Цифровая обработка изображений

Лекция 6

Геометрические преобразования изображений. Методы интерполяции.

Сафонов Илья Владимирович

ilia.safonov@gmail.com

2025

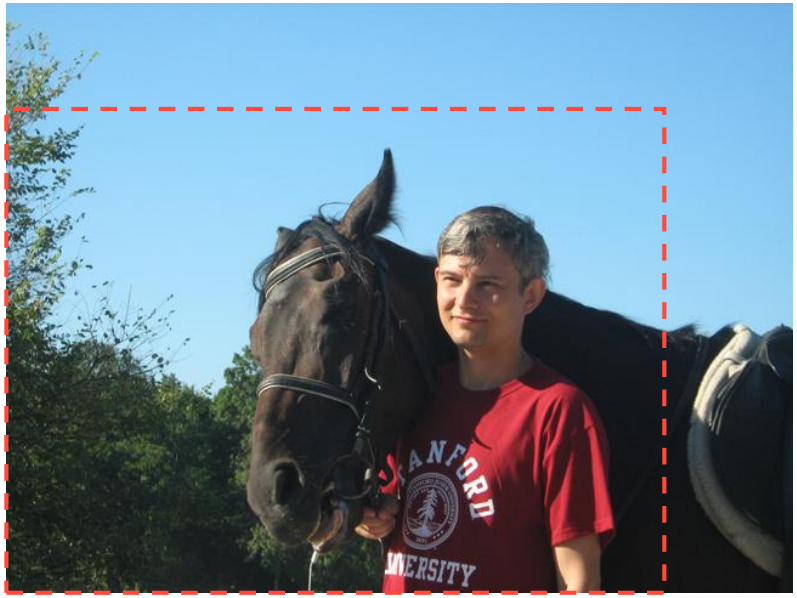
Геометрические преобразования

- Кадрирование
- Масштабирование
- Поворот
- Зеркальные отражения
- Скос
- Коррекция геометрических искажений
- Переформатирование (retargeting)

Аффинные преобразования

Кадрирование

Кадрирование (*cropping*) – вырезание из изображения прямоугольного фрагмента. Выполняют с целью выделения требуемой части изображения, изменения соотношения сторон (*aspect ratio*) и/или улучшения композиции.



Однородные координаты

Аффинные преобразования на плоскости задаются с помощью матрицы 3x3 трансформации в однородных (homogeneous) координатах:

$$T = \begin{bmatrix} a & b & 0 \\ c & d & 0 \\ t_x & t_y & 1 \end{bmatrix}$$

Аффи́нное преобразование - отображение плоскости или пространства в себя, при котором параллельные прямые переходят в параллельные прямые, пересекающиеся - в пересекающиеся, скрещивающиеся - в скрещивающиеся

Сама трансформация может быть записана двумя способами:

1. $[x' \quad y' \quad 1] = [x \quad y \quad 1]T$ — Вектор-строка координат умножается на матрицу T

2. $\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = T^t \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$ — Матрица T^t умножается на вектор-столбец координат

$$x' = ax + cy + t_x$$
$$y' = bx + dy + t_y$$

x, y – исходные координаты точки
x', y' – преобразованные координаты

Разница двух способов преобразования

Если выполняется несколько преобразований, то итоговую матрицу можно получить путём перемножения матриц каждого из преобразований. Здесь есть отличия в зависимости от способа трансформации.

1. $[x' \quad y' \quad 1] = [x \quad y \quad 1]T$
2. $\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = T^t \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$

Пусть последовательно выполняются преобразования, задаваемые матрицами T_1(в виде как приведено на предыдущем слайде), T_2 и T_3. Тогда итоговая матрица T для двух способов трансформации:

1. $T = T_1 T_2 T_3$
2. $T = T_3^t T_2^t T_1^t$

Поскольку матричное перемножение некоммутативно, то важно последовательно производить перемножение слева направо.

Общая схема преобразования

1. Координаты угловых пикселов исходного изображения умножаются на прямую матрицу трансформации для нахождения координат угловых пикселов и размера нового изображения:

$$\begin{bmatrix} x'_i & y'_i & 1 \end{bmatrix} = \begin{bmatrix} x_i & y_i & 1 \end{bmatrix} \times \begin{bmatrix} a & b & 0 \\ c & d & 0 \\ t_x & t_y & 1 \end{bmatrix}$$

2. Координаты каждого пиксела нового изображения умножают на обратную матрицу трансформации для получения координаты соответствующего ему пиксела в исходном изображении:

$$\begin{bmatrix} x & y & 1 \end{bmatrix} = \begin{bmatrix} x' & y' & 1 \end{bmatrix} \times \begin{bmatrix} a & b & 0 \\ c & d & 0 \\ t_x & t_y & 1 \end{bmatrix}^{-1}$$

3. Если координата попала в пределы габаритов исходного изображения, то значение пиксела в новом изображении определяют с помощью интерполяции по соседним пикселам исходного изображения.

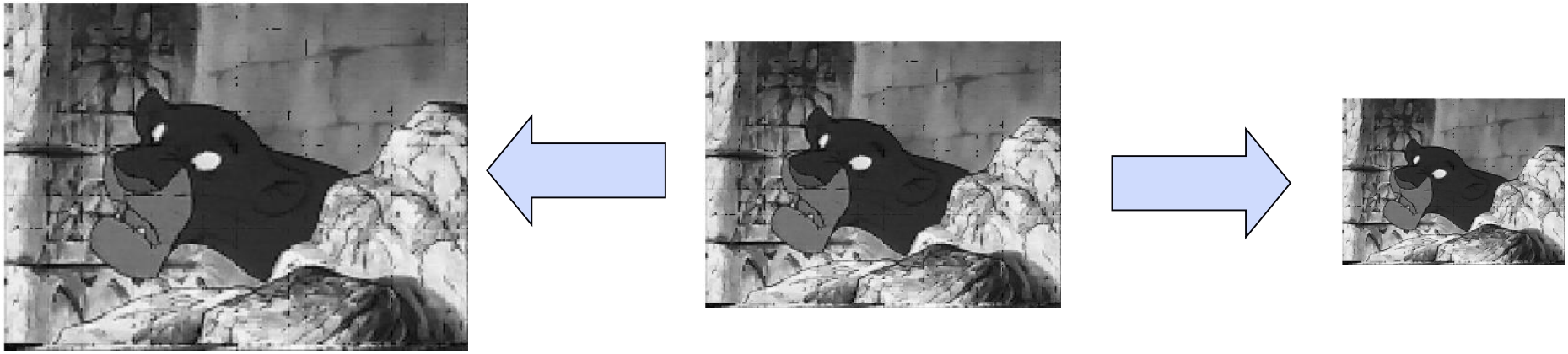
Масштабирование

Масштабирование (*scaling*) – изменение размеров (*resizing*) изображения. Различают увеличение размеров (*upsizing*) и уменьшение размеров (*downsizing*).

Матрицы преобразования для масштабирования:

Прямая $\begin{bmatrix} Sx & 0 & 0 \\ 0 & Sy & 0 \\ 0 & 0 & 1 \end{bmatrix}$

Обратная $\begin{bmatrix} \frac{1}{Sx} & 0 & 0 \\ 0 & \frac{1}{Sy} & 0 \\ 0 & 0 & 1 \end{bmatrix}$



Поворот (1)

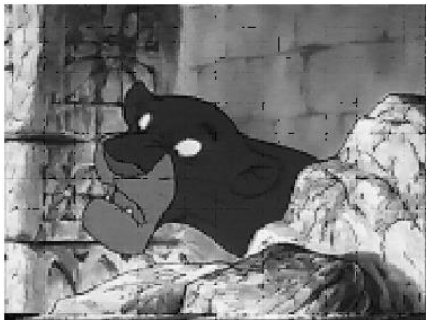
Поворот на угол φ против часовой стрелки вокруг начала координат выполняют с помощью следующей матрицы преобразования:

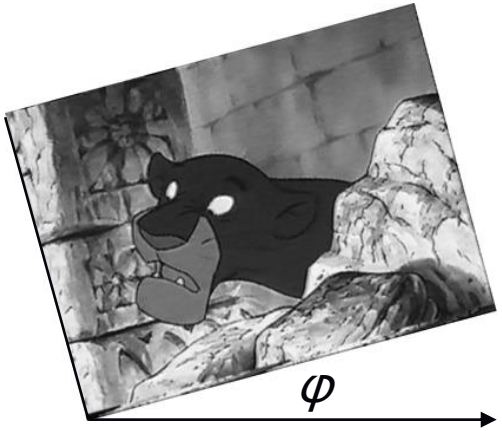
Прямая

$$\begin{bmatrix} \cos\varphi & \sin\varphi & 0 \\ -\sin\varphi & \cos\varphi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Обратная

$$\begin{bmatrix} \cos\varphi & -\sin\varphi & 0 \\ \sin\varphi & \cos\varphi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$





Для повышения скорости обработки поворот на углы кратные 90° обычно реализуют как частные случаи.

Поворот (2)

Часто выполняют поворот вокруг центра изображения (x_c, y_c). В этом случае прямая матрица преобразования есть результат произведения матрицы переноса начала координат в центр изображения, матрицы поворота и матрицы переноса начала координат в начальную позицию:

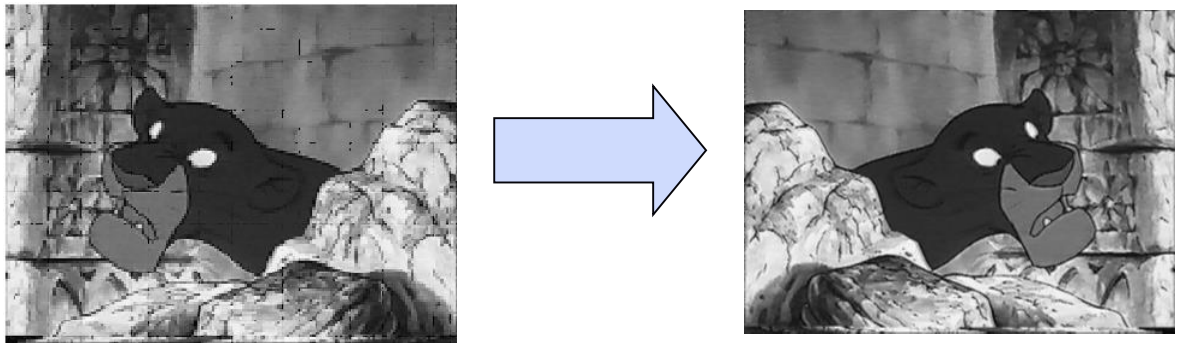
$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -x_c & -y_c & 1 \end{bmatrix} \times \begin{bmatrix} \cos\varphi & \sin\varphi & 0 \\ -\sin\varphi & \cos\varphi & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ x_c & y_c & 1 \end{bmatrix}$$

Иногда требуется выполнить поворот на произвольный угол в том же буфере памяти. Такие алгоритмы называют *in place* или *in situ*. В этом случае можно заменить 1 поворот на 2 последовательных сдвига (*shear*, *skew*). Например, возможна такая декомпозиция обратной матрицы поворота:

$$\begin{bmatrix} \cos\varphi & -\sin\varphi & 0 \\ \sin\varphi & \cos\varphi & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \cos\varphi & 0 & 0 \\ \sin\varphi & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 1 & -\tan\varphi & 0 \\ 0 & \sec\varphi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

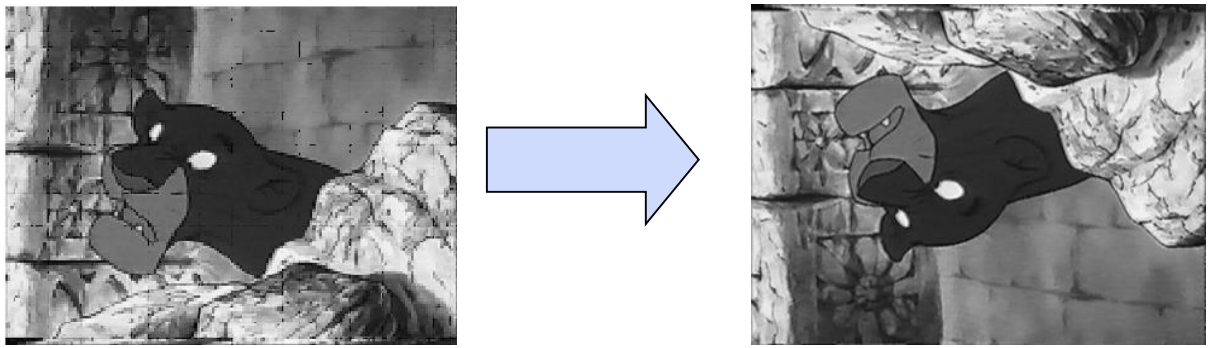
Зеркальное отражение

Зеркальное отражение (*flip*) относительно вертикальной оси:



$$T = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Зеркальное отражение (*flip*) относительно горизонтальной оси:

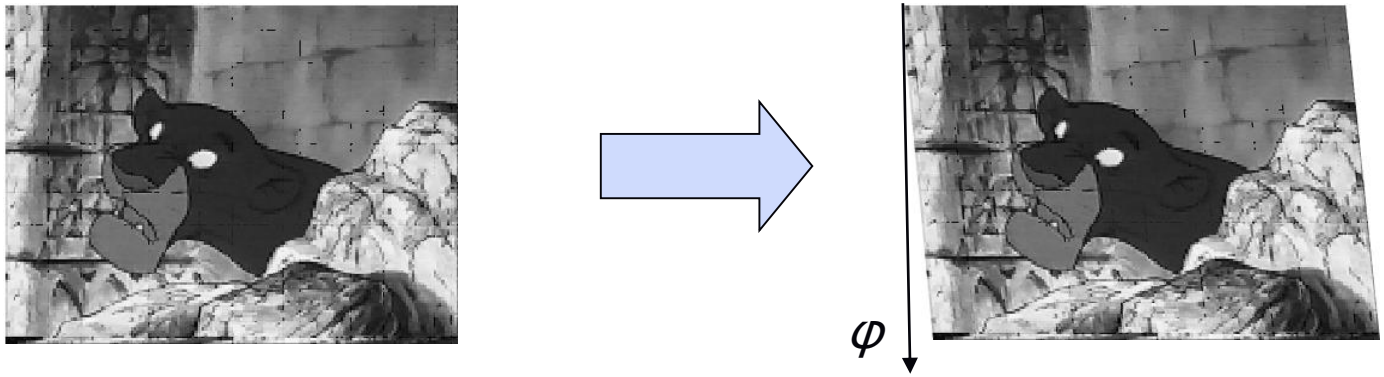


$$T = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Обычно реализуют как частные случаи.

Скос

Скос (*skew, shear*) по X:

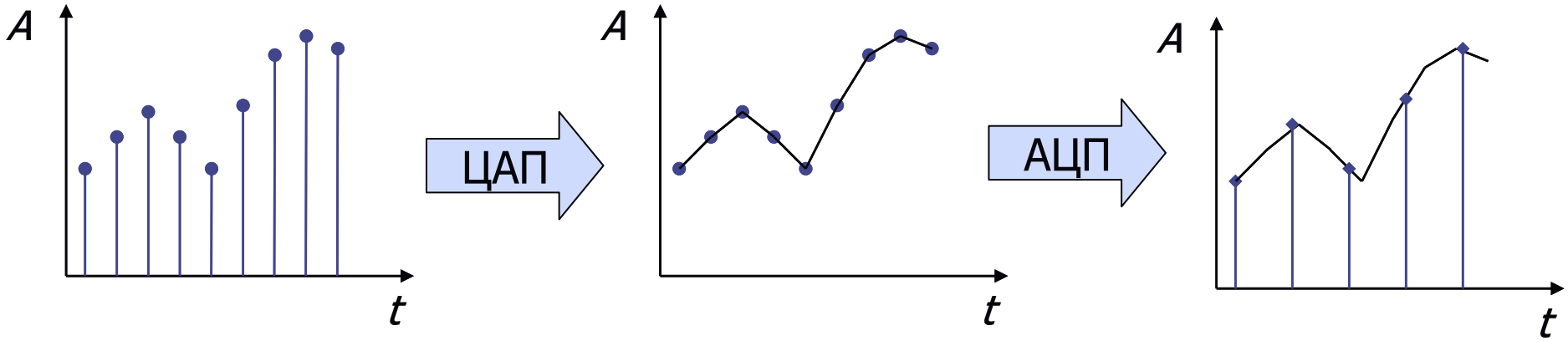


$$T = \begin{bmatrix} 1 & 0 & 0 \\ tg\varphi & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Скос по Y получается транспонированием матрицы
Возможен скос одновременно по X и Y.

Перевыборка / Передискретизация

Передискретизация (перевыборка, resampling) – изменение частоты сигнала, т.е. изменение количества отсчётов. Передискретизация подразумевает цифро-аналоговое преобразование исходного сигнала и новое аналого-цифровое преобразование с другой частотой. При цифро-аналоговом преобразовании выполняется интерполяция для восстановления непрерывного сигнала из дискретного.



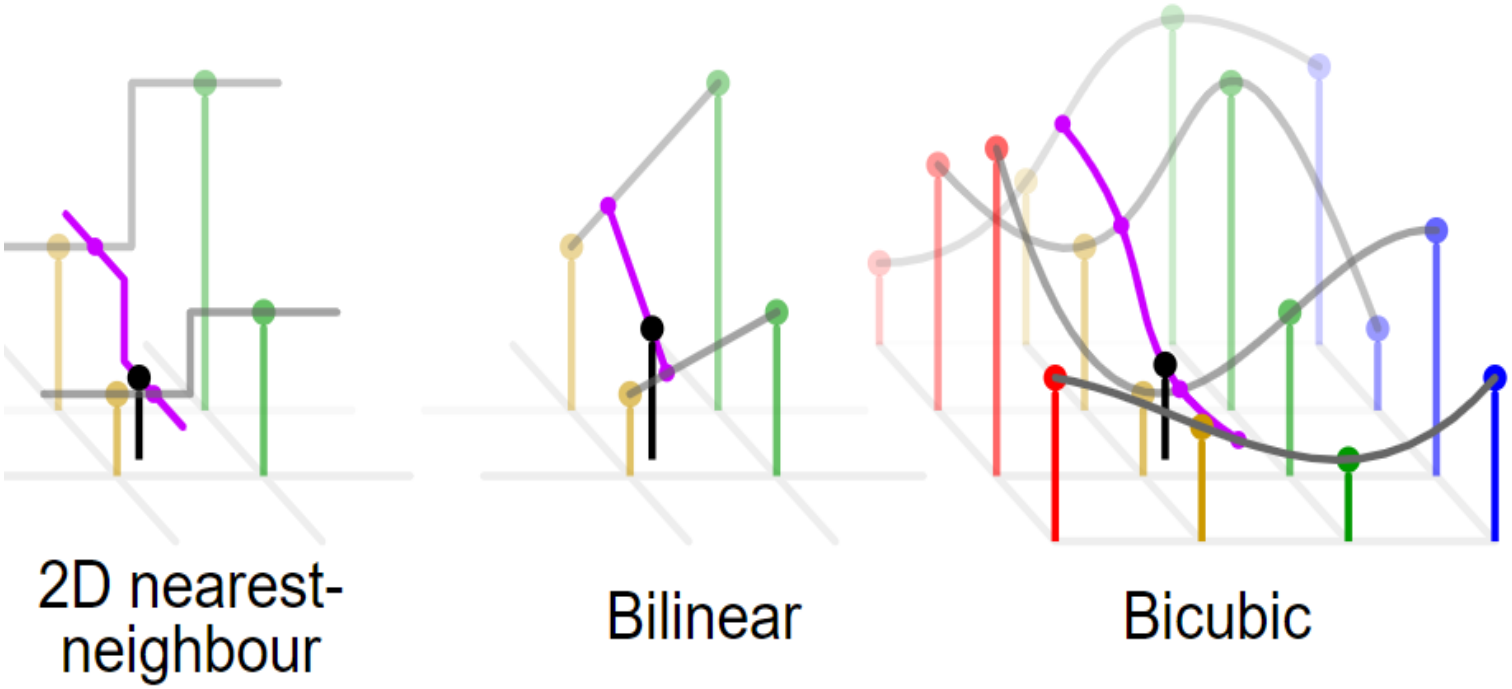
Уменьшение количества отсчётов сигнала также называют децимацией.

Изменение количества отсчётов у изображения соответствует масштабированию (изменению размеров). Различают увеличение изображений (upsizing, upsampling) и уменьшение изображений (downsizing, downsampling).

Способы интерполяции изображений

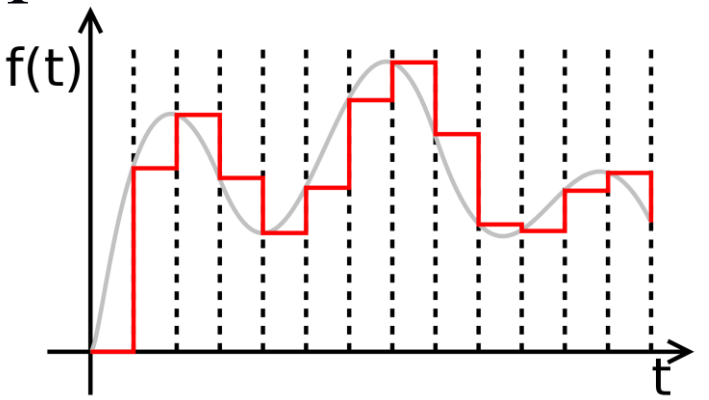
Для интерполяции изображений применяются 2 подхода:

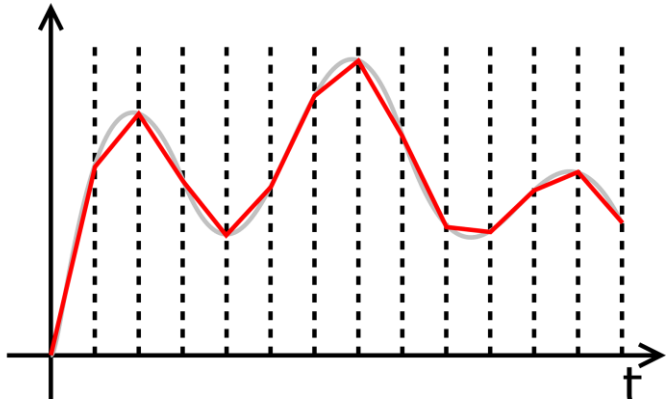
- 1D интерполяция способами для сигналов сначала по строкам, а затем по столбцам
- 2D интерполяция двумерными функциями:
  - ближайшего соседа или по ближайшему соседу (nearest neighbour)
  - билинейная (Bilinear)
  - бикубическая (bicubic)

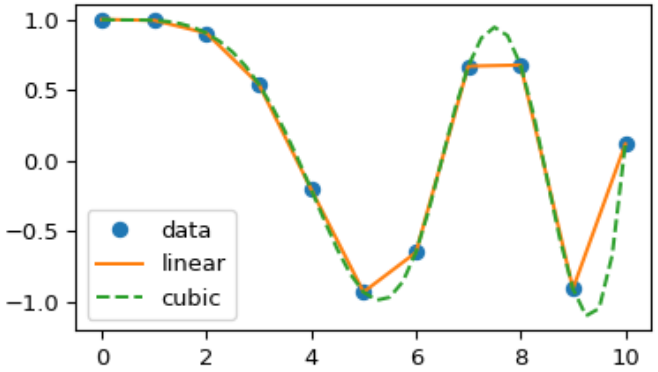


Способы интерполяции одномерных сигналов

1. Интерполяция функциями
   - 0-го порядка (по текущему или ближайшему отсчёту)
   - 1-го порядка, т.е. кусочно-линейная интерполяция
   - 2-го порядка (квадратичная), полиномами 2-й степени
   - 3-го порядка (кубическая), сплайнами или полиномами 3-й степени
2. Фильтры, построенные, исходя из критерия близости их АЧХ к АЧХ идеального ФНЧ
   - Sinc интерполяция
   - фильтры Чебышева







Sinc интерполяция (формула Уиттекера — Шеннона)

При соблюдении граничных условий непрерывный сигнал *x(t)* может быть восстановлен из дискретного *x[n]*:

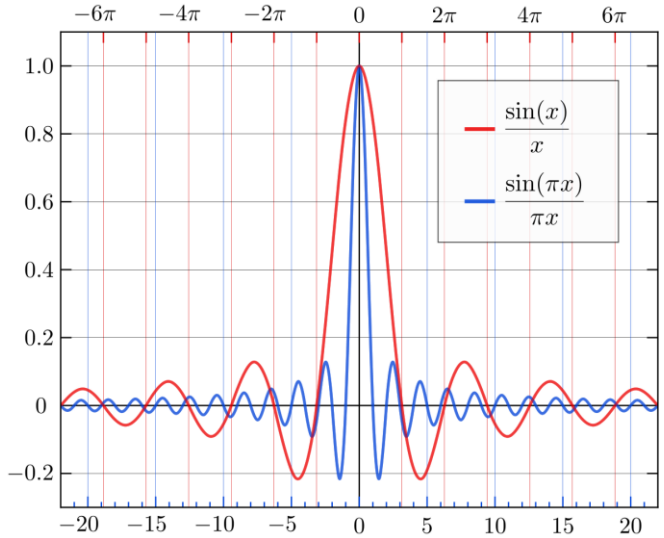
$$x(t) = \sum_{n=-\infty}^{\infty} x[n] \cdot \mathrm{sinc}\left(\frac{t - nT}{T}\right)$$

где T – период дискретизации (величина обратная частоте).

Ограничения применимости:

1. Результат преобразования Фурье *x(t)* для частот больше B равно 0.
2. Частота дискретизации должна, как минимум, в 2 раза превышать диапазон частот сигнала, т.е. $1/2B > T$.

На практике граничные условия часто не выполняются, кроме того, есть только конечное число отсчётов сигнала.



В обработке сигналов используется нормированная функция sinc(x):

$$\mathrm{sinc}(x) = \begin{cases} \dfrac{\sin(\pi x)}{\pi x} & ; \quad x \neq 0 \\ 1 & ; \quad x = 0 \end{cases}$$

В математике ненормированная функция sinc(x) определяется как:

$$\mathrm{sinc}(x) = \begin{cases} \dfrac{\sin(x)}{x} & ; \quad x \neq 0 \\ 1 & ; \quad x = 0 \end{cases}$$

Пример Sinc интерполяции

```
#include <math.h>

double sinc(double x)
{
    return (x == 0. ? 1. : sin(M_PI * x) / (M_PI * x));
}

void SincInterpolation1D( const double* pbIn, int lSizeIn,
        double *pbOut, int lSizeOut )
{
    double st = (double)(lSizeIn - 1) / (lSizeOut - 1);
    for ( int i = 0; i < lSizeOut; ++i )
    {
        double sum = 0.;
        for ( int j = 0; j < lSizeIn; ++j )
        {
            sum += pbIn[j] * sinc(st * i - j);
        }
        pbOut[i] = sum;
    }
}
```

Фильтр Ланцоша (Lánczos)

Применение Sinc фильтра для увеличения изображений ведёт к возникновению артефактов (ringing, «звона») вдоль резких границ. Это происходит из-за нарушения ограничений. Если применять фильтр Sinc в окне, то «звон» не возникает. Удачный вариант фильтра с оконной функцией в виде предложил Ланцош.

Для сигнала с дискретными отсчётами s_i (i - целые) интерполированные значения $S(x)$ для действительных значений x получают с помощью свёртки с ядром Ланцоша:

$$S(x) = \sum_{i=\lfloor x \rfloor - a + 1}^{\lfloor x \rfloor + a} s_i L(x - i)$$

$$L(x) = \begin{cases} 1 & \text{если } x = 0, \\ \dfrac{a \sin(\pi x) \sin(\pi x / a)}{\pi^2 x^2} & \text{если } -a \le x < a \text{ и } x \ne 0, \\ 0 & \text{иначе.} \end{cases}$$

На практике параметр a выбирают равным 2 или 3.

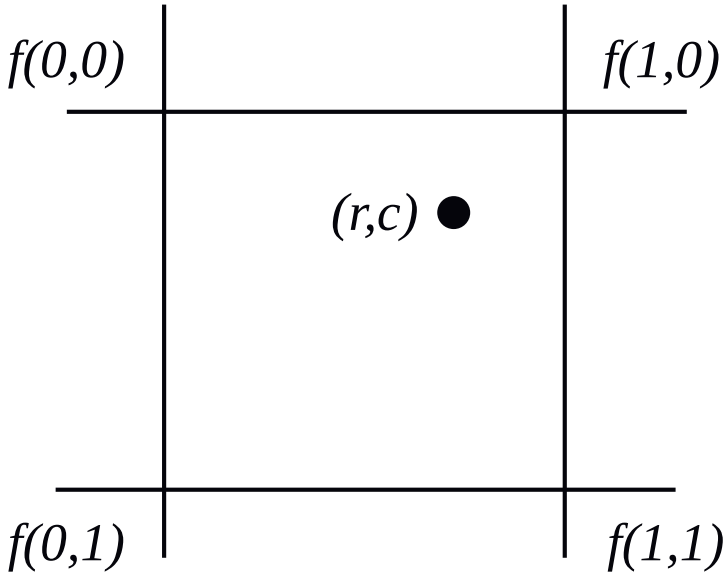
Изображения сначала интерполируют по строкам, а затем по столбцам.

Интерполяция “по ближайшему соседу”

В интерполяции “по ближайшему соседу” результатом является значение ближайшего пиксела из четырёх соседних.

Значения в вершинах “единичного квадрата” известны:

$f(0,0) = I([c], [r])$
$f(1,0) = I([c]+1, [r])$
$f(0,1) = I([c], [r]+1)$
$f(1,1) = I([c]+1, [r]+1)$



Требуется найти интерполированное значение I' внутри квадрата в точке (r,c).

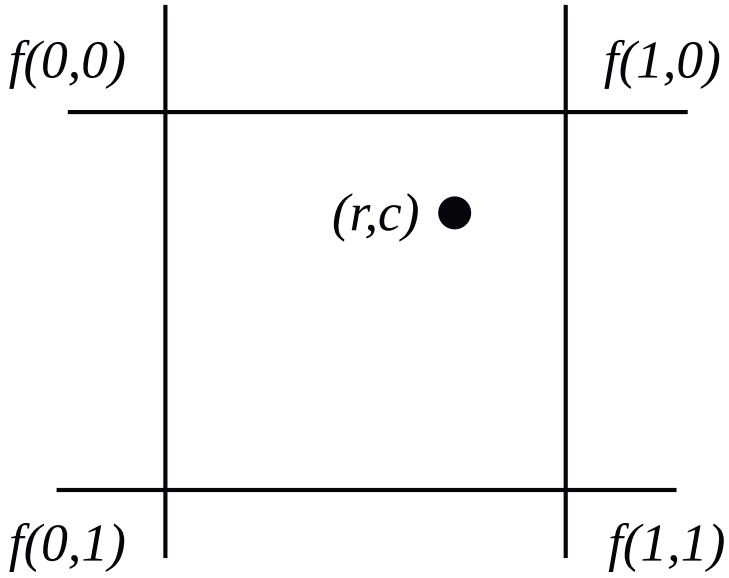
$I'(r,c) = f([c+0.5]-[c], [r+0.5]-[r])$,
где $[]$ – операция взятия целой части

Билинейная интерполяция

Билинейная – это интерполяция по билинейной поверхности, построенной по значениям 4-х ближайших пикселов.
Может быть вычислена как 3 одномерные интерполяции 1-го порядка.

Значения в вершинах “единичного квадрата” известны:

$f(0,0) = I([c], [r])$
$f(1,0) = I([c]+1, [r])$
$f(0,1) = I([c], [r]+1)$
$f(1,1) = I([c]+1, [r]+1)$



Требуется найти интерполированное значение I' внутри квадрата в точке (r,c).

$I'(r,c) = f(x,y) = f(0,0)(1-x)(1-y) + f(1,0)x(1-y) + f(0,1)(1-x)y + f(1,1)xy$
где $x = c – [c]$; $y = r – [r]$; $[]$ – операция взятия целой части

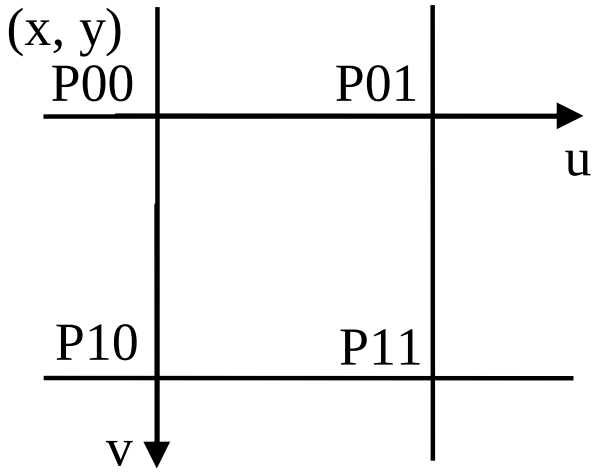
Пример масштабирования изображения (1)

Пример масштабирования изображения 8 bpp с помощью билинейной интерполяции.

```
void ResizeBilinear( const unsigned char* pbIn, int lWidthIn,
        int lHeightIn, unsigned char *pbOut, int lWidthOut, int lHeightOut )
{
    for( int i = 0; i < lHeightOut; ++i )
    {
        double yy = (double)i*(double)lHeightIn/lHeightOut;
        int y = (int)yy;            // целая часть yy
        double u = yy - (double)y;  // дробная часть yy
        for( int j = 0; j < lWidthOut; ++j )
        {
            double xx = (double)j*(double)lWidthIn/lWidthOut;
            int x = (int)xx ;           // целая часть xx
            double v = xx - (double)x; // дробная часть xx
```

Пример масштабирования изображения (2)

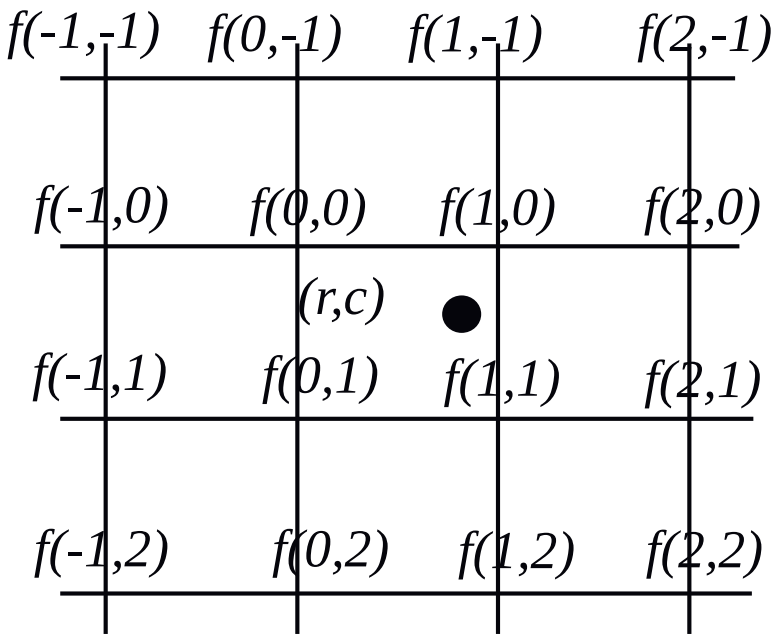
```
// значения соседних пикселов
            int lP00 = pbIn[y*lWidthIn + x],
                lP01 = pbIn[y*lWidthIn + x + 1],
                lP10 = pbIn[(y+1)*lWidthIn + x],
                lP11 = pbIn[(y+1)*lWidthIn + x + 1];
            // билинейная интерполяция
            pbOut[i*lWidthOut+j] = (unsigned char)( (1. - u)*(1. - v)*lP00 +
                u*(1. - v)*lP10 + v*(1. - u)*lP01 +
                u*v*lP11 );
        }
    }
}
```



Бикубическая интерполяция (1)

Бикубическая интерполяция выполняется по 16 соседним точкам. Существует несколько способов бикубической интерполяции.
Один из популярных – расширение одномерной кубической интерполяции на двумерную регулярную сетку:

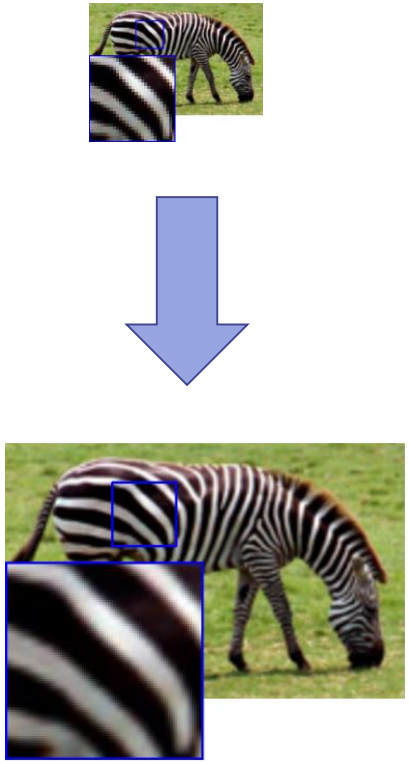
Значения в вершинах:
$f(-1,-1) = I([c]-1, [r]-1)$
$f(0,-1) = I([c], [r]-1)$
$f(1,-1) = I([c]+1, [r]-1)$
$f(2,-1) = I([c]+2, [r]-1)$
$f(-1,0) = I([c]-1, [r])$
$f(0,0) = I([c], [r])$
…
$f(2,2) = I([c]+2, [r]+2)$



$I'(c,r)=$
$f(x,y)=b_1f(0,0)+b_2f(0,1)+b_3f(1,0)+b_4f(1,1)+b_5f(0,-1)+b_6f(-1,0)+b_7f(1,-1)+b_8f(-1,1)$
$+b_9f(0,2)+b_{10}f(2,0)+b_{11}f(-1,-1)+b_{12}f(1,2)+b_{13}f(2,1)+b_{14}f(-1,2)+b_{15}f(2,-1)+b_{16}f(2,2)$
где $x = c – [c]$; $y = r – [r]$; $[]$ – операция взятия целой части

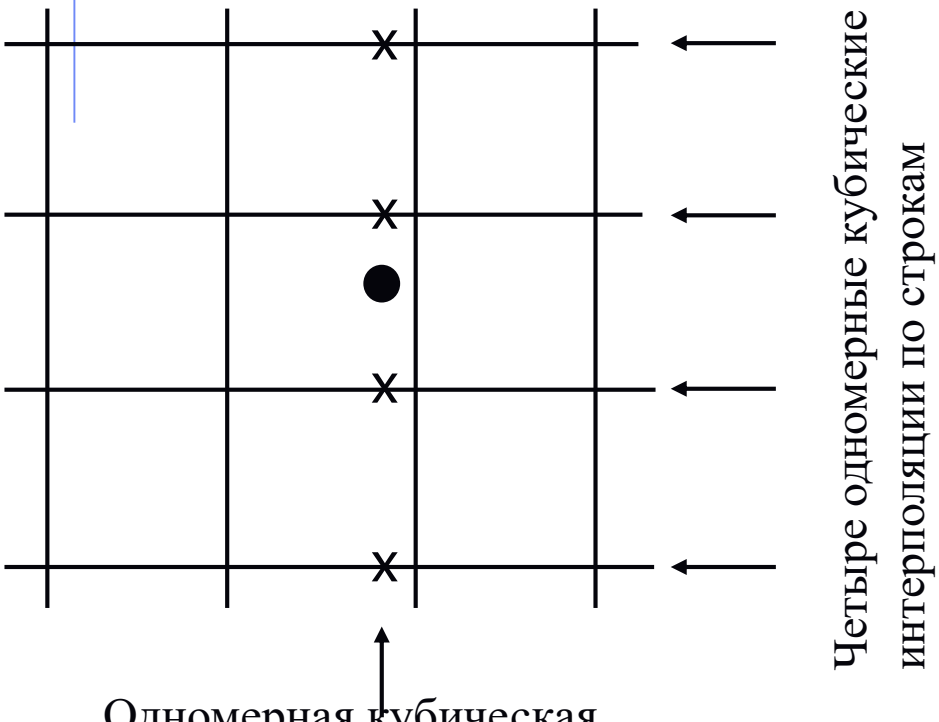
Бикубическая интерполяция (2)

$b_1 = (x-1)(x-2)(x+1)(y-1)(y-2)(y+1)/4$
$b_2 = -x(x-2)(x+1)(y-1)(y-2)(y+1)/4$
$b_3 = -(x-1)(x-2)(x+1)y(y-2)(y+1)/4$
$b_4 = x(x-2)(x+1)y(y-2)(y+1)/4$
$b_5 = -x(x-1)(x-2)(y-1)(y-2)(y+1)/12$
$b_6 = -(x-1)(x-2)(x+1)y(y-1)(y-2)/12$
$b_7 = x(x-1)(x-2)y(y-2)(y+1)/12$
$b_8 = x(x-2)(x+1)y(y-1)(y-2)/12$
$b_9 = x(x-1)(x+1)(y-1)(y-2)(y+1)/12$
$b_{10} = (x-1)(x-2)(x+1)y(y-1)(y+1)/12$
$b_{11} = x(x-1)(x-2)y(y-1)(y-2)/36$
$b_{12} = -x(x-1)(x+1)y(y-2)(y+1)/12$
$b_{13} = -x(x-2)(x+1)y(y-1)(y+1)/12$
$b_{14} = -x(x-1)(x+1)y(y-1)(y-2)/36$
$b_{15} = -x(x-1)(x-2)y(y-1)(y+1)/36$
$b_{16} = x(x-1)(x+1)y(y-1)(y+1)/36$



Кубическая свёртка

Один из вариантов бикубической интерполяции - кубическая свертка (*cubic convolution*), которая состоит из пяти одномерных кубических интерполяций, четыре из которых делаются по строкам около интерполируемой точки, а пятый делается по четырем интерполированным значениям.



$$f(x) = f(-1)u(x-1)+f(0)u(x)+f(1)u(x+1)+f(2)u(x+2)$$

$$u(s) = \begin{cases} (a+2)|s|^3 - (a+3)|s|^2 + 1, & 0 < |s| \le 1 \\ a|s|^3 - 5a|s|^2 + 8a|s| - 4a, & 1 < |s| \le 2 \\ 0, & 2 < |s| \end{cases}$$

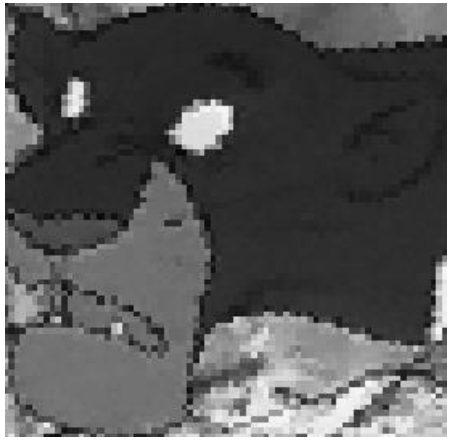
Параметр $a=-0.5$ у автора алгоритма. Встречаются также реализации с $a=-0.75$ и $a=-1$.

Сравнение способов интерполяции

скорость

качество

по ближайшему соседу
(0-го порядка)



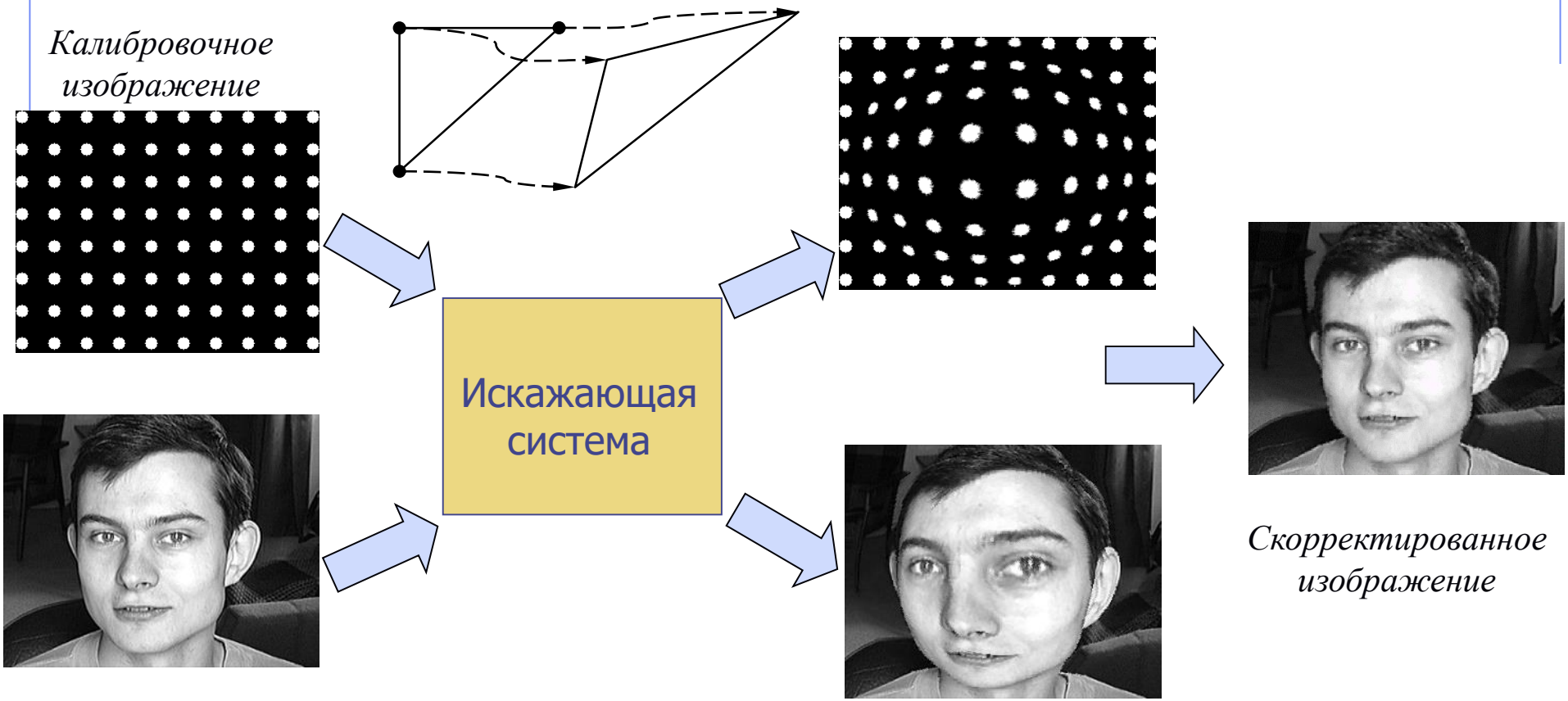
билинейная

бикубическая

Увеличенные фрагменты интерполированных изображений

Коррекция дисторсии (1)

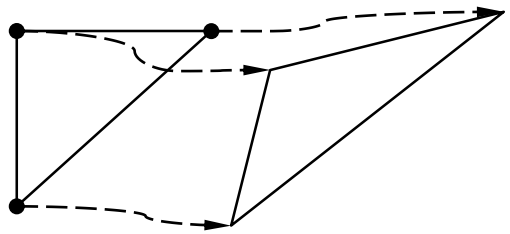
Изображения подвергнутые геометрическим искажениям (дисторсии) могут быть скорректированы на основе данных калибровки.



Аналогичным образом изображения можно исказить (warping).

Коррекция дисторсии (2)

Существует взаимно однозначное соответствие между координатами треугольников исходного и искаженного калибровочного изображений, что позволяет для каждого треугольника определить матрицу трансформации путем решения системы линейных уравнений.

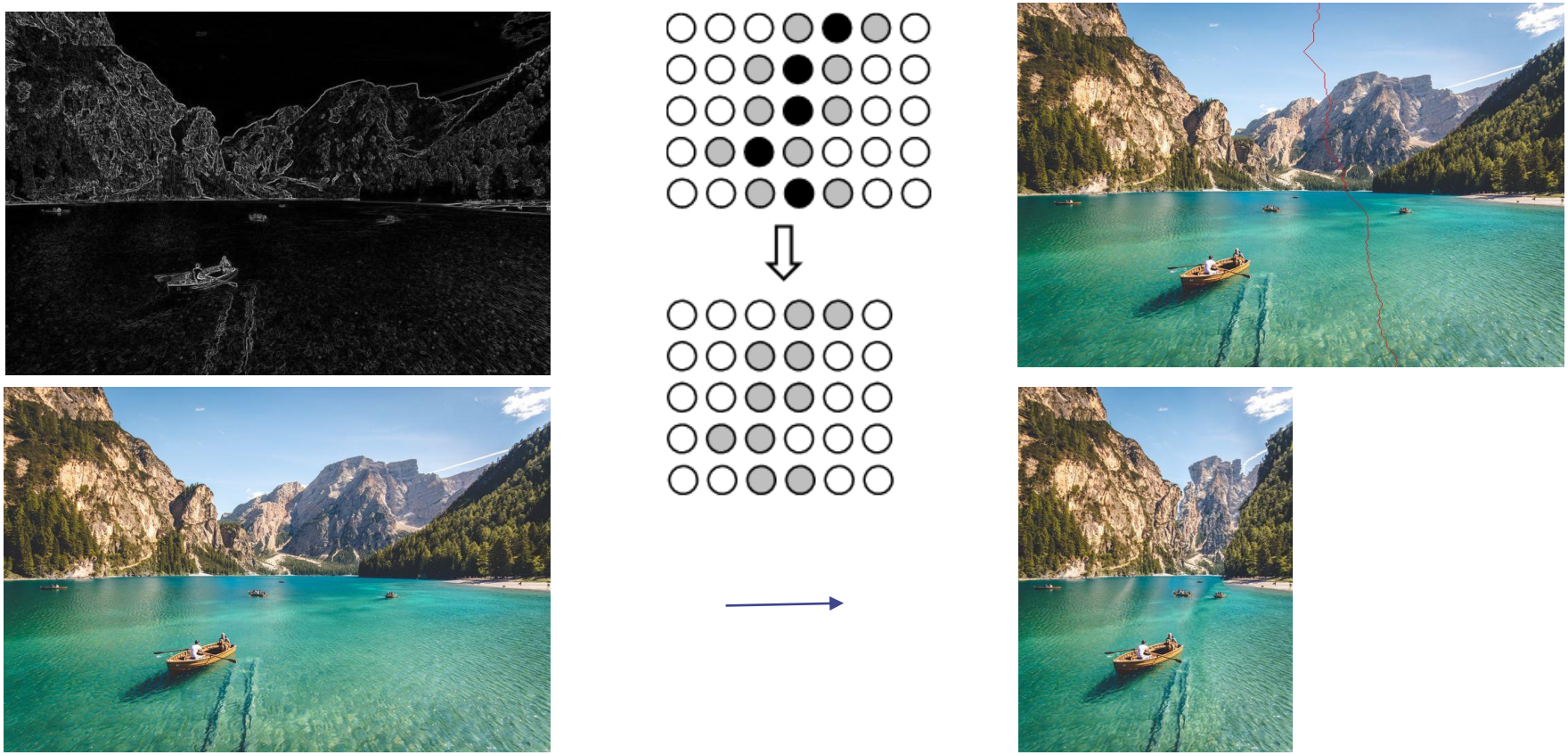


$$\begin{bmatrix} x'_i & y'_i & 1 \end{bmatrix} = \begin{bmatrix} x_i & y_i & 1 \end{bmatrix} \times \begin{bmatrix} a & b & 0 \\ c & d & 0 \\ t_x & t_y & 1 \end{bmatrix}, \quad i = 0,1,2.$$

$$\begin{cases} a = \dfrac{-x'_1 \cdot y_0 + x'_1 \cdot y_2 + y_1 \cdot x'_0 - y_1 \cdot x'_2 + x'_2 \cdot y_0 - y_2 \cdot x'_0}{x_0 \cdot y_1 - x_0 \cdot y_2 - y_0 \cdot x_1 + y_0 \cdot x_2 - x_2 \cdot y_1 + y_2 \cdot x_1} \\ b = \dfrac{y'_2 \cdot y_0 - y_2 \cdot y'_0 - y'_1 \cdot y_0 + y'_1 \cdot y_2 + y_1 \cdot y'_0 - y_1 \cdot y'_2}{x_0 \cdot y_1 - x_0 \cdot y_2 - y_0 \cdot x_1 + y_0 \cdot x_2 - x_2 \cdot y_1 + y_2 \cdot x_1} \\ c = \dfrac{x'_0 \cdot x_2 - x'_0 \cdot x_1 + x_0 \cdot x'_1 - x_0 \cdot x'_2 + x'_2 \cdot x_1 - x_2 \cdot x'_1}{x_0 \cdot y_1 - x_0 \cdot y_2 - y_0 \cdot x_1 + y_0 \cdot x_2 - x_2 \cdot y_1 + y_2 \cdot x_1} \\ d = \dfrac{y'_0 \cdot x_2 - y'_0 \cdot x_1 - x_0 \cdot y'_2 + x_0 \cdot y'_1 - y'_1 \cdot x_2 + x_1 \cdot y'_2}{x_0 \cdot y_1 - x_0 \cdot y_2 - y_0 \cdot x_1 + y_0 \cdot x_2 - x_2 \cdot y_1 + y_2 \cdot x_1} \\ t_x = \dfrac{y_1 \cdot x_0 \cdot x'_2 - x'_2 \cdot y_0 \cdot x_1 + x'_1 \cdot y_0 \cdot x_2 - y_1 \cdot x'_0 \cdot x_2 + y_2 \cdot x'_0 \cdot x_1 - x'_1 \cdot x_0 \cdot y_2}{x_0 \cdot y_1 - x_0 \cdot y_2 - y_0 \cdot x_1 + y_0 \cdot x_2 - x_2 \cdot y_1 + y_2 \cdot x_1} \\ t_y = \dfrac{-y_2 \cdot x_0 \cdot y'_1 + y'_1 \cdot y_0 \cdot x_2 - y'_2 \cdot y_0 \cdot x_1 + y_2 \cdot y'_0 \cdot x_1 - y_1 \cdot y'_0 \cdot x_2 + y'_2 \cdot x_0 \cdot y_1}{x_0 \cdot y_1 - x_0 \cdot y_2 - y_0 \cdot x_1 + y_0 \cdot x_2 - x_2 \cdot y_1 + y_2 \cdot x_1} \end{cases}$$

Переформатирование (1)

Переформатированием (retargeting) называется визуально незаметное изменение/добавление локальных фрагментов изображения с целью изменить его размеры, часто чтобы изменить соотношение сторон.
Seam carving является одним из наиболее известных алгоритмов retargeting.

Переформатирование (2)

В seam carving ФВЧ строится карта “стоимости/энергии” и с помощью динамического программирования определяется цепочка пикселей между противоположными сторонами, имеющая наименьшую стоимость. Эта цепочка либо удаляется (изображение уменьшается по данному измерению), либо вдоль неё добавляется новая цепочка пикселей (изображение увеличивается). Путём итераций получают требуемый размер.